



دانشگاه اصفهان
دانشکده مهندسی کامپیوتر

درس مهندسی اینترنت

فاز دو پروژه «توزیع مبتنی بر Multicast در GNS3»

عنوان گزارش:

پیاده‌سازی برنامه‌ی Canvas اشتراکی Real-Time مبتنی بر UDP Multicast با
پشتیبانی از IGMP Proxy و حالت PIM

دانشجو:

سپهر رجبی

۴۰۱۳۶۲۳۰۰۸

پاییز ۱۴۰۴

۱-۲ مقدمه و هدف فاز دوم

فاز دوم پروژه روی همان زیرساخت فاز اول سوار می‌شود، با این تفاوت که این بار به جای یک تست ساده‌ی «ارسال/دریافت»، یک برنامه‌ی چندکاربره‌ی تعاملی و گرافیکی پیاده‌سازی می‌شود که همه‌ی کاربران یک Canvas مشترک دارند و هر عملیاتی که یک کاربر انجام می‌دهد – مثل کشیدن خط، رسم شکل، نوشتن متن، پاک کردن یا تغییر پس‌زمینه – برای سایر کاربران نیز به صورت Real-Time پخش و نمایش داده می‌شود.

در این فاز، هدف دقیقاً تحقق بندهای زیر از صورت مسئله است:

- **برنامه‌ی واحد برای هر میزبان (هم فرستنده و هم گیرنده).**
- استفاده از **UDP + Multicast** برای ارسال رویدادهای گرافیکی (Canvas events).
- **قابل تنظیم بودن** Multicast Group و Port و TTL.
- **عضویت در گروه** و دریافت بسته‌ها و **پاژس کردن** داده‌ها و ترسیم روی بوم.
- **عدم وجود سرور** و همگام‌سازی صرفاً با چندپخش.

نکته‌ی مهم درباره‌ی فاز اول و کدهای Scapy:

در فاز اول علاوه بر نسخه‌ی مبتنی بر socket، یک نسخه‌ی دقیق‌تر و قابل مشاهده‌تر در سطح لایه‌ی دو با Scapy نیز پیاده‌سازی شده است تا:

- تولید بسته‌های UDP Multicast و پاسخ Unicast کاملاً با sendp و ساخت دستی Ether/IP/UDP/Raw انجام شود؛
- IGMP JOIN/LEAVE به صورت صریح (با Router Alert) ساخته و ارسال شود تا تحلیل در Wireshark شفاف‌تر گردد؛
- برای پاسخ‌های Unicast به میزبان سمت مقابل، ARP برای یافتن Next-Hop MAC استفاده شود.

کد کامل این دو فایل در پوشه‌ی پروژه قرار دارد: Phase1_Scapy_Sender.py و Phase1_Scapy_Receiver.py. (در این گزارش از کپی‌کردن کامل کدها داخل PDF خودداری شده و به جای آن روی طراحی و تحلیل تمرکز شده است.)

برای تکمیل مستندات و اتصال منطقی دو فاز، چند تصویر نمونه از اجرای نسخه‌ی Scapy فاز اول نیز در این گزارش درج شده است (صرفاً به عنوان یادآوری و نشان‌دادن صحت مسیر

چندپخشی). توضیحات جزئی‌تر این بخش در گزارش فاز اول آمده و در این فاز از تکرار بیش از حد آن خودداری می‌شود.

```

237     try:
238         while True:
239             msg = "Hello multicast #{} from {}".format(i, src_ip).encode("utf-8")
240             pkt = (
241                 Ether(src=src_mac, dst=m_dst_mac) /
242                 IP(src=src_ip, dst=args.group, ttl=
243                 UDP(sport=args.port, dport=args.port) /
244                 Raw(load=msg)
245             )
246             sendp(pkt, iface=iface, verbose=False)
247             print("[SENDER][MCAST] >>> {}".format
248             sniff(iface=iface, filter=bpf, prn=han
249             i += 1
250             time.sleep(float(args.interval))
251
252     except KeyboardInterrupt:
253         print("\n[SENDER] Ctrl+C detected, exiting.")
254
255 if __name__ == "__main__":
256     main()
257
258
259
260

```

```

[SENDER] <<< PING from 10.0.2.10:5000
[SENDER] >>> PONG to 10.0.2.11
[SENDER] <<< PING from 10.0.2.11:5000
[SENDER][MCAST] >>> 'Hello multicast #6 from 10.0.1.10'
[SENDER] >>> PONG to 10.0.2.11
[SENDER] <<< PING from 10.0.2.11:5000
[SENDER] >>> PONG to 10.0.2.11
[SENDER] <<< PING from 10.0.2.11:5000
[SENDER][MCAST] >>> 'Hello multicast #7 from 10.0.1.10'
[SENDER] >>> PONG to 10.0.2.11
[SENDER] <<< PING from 10.0.2.11:5000
[SENDER] >>> PONG to 10.0.2.11
[SENDER] <<< PING from 10.0.2.11:5000
[SENDER][MCAST] >>> 'Hello multicast #8 from 10.0.1.10'
[SENDER] >>> PONG to 10.0.2.11
[SENDER] <<< PING from 10.0.2.11:5000
[SENDER] >>> PONG to 10.0.2.11
[SENDER] <<< PING from 10.0.2.11:5000
[SENDER][MCAST] >>> 'Hello multicast #9 from 10.0.1.10'
[SENDER] >>> PONG to 10.0.2.11
[SENDER] <<< PING from 10.0.2.11:5000
[SENDER] >>> PONG to 10.0.2.11
[SENDER] <<< PING from 10.0.2.11:5000
[SENDER] >>> PONG to 10.0.2.11

```

شکل ۱: فاز اول – نسخه‌ی Scapy – اجرای Sender روی PC1 و ارسال دوره‌ای UDP Multicast به همراه پاسخ‌دهی PONG به PING‌ها.

```

290     if text.upper() == "PONG":
291         print("[RECEIVER] <<< Received unicast PONG from sender {}".format(ip.src))
292     else:
293         print("[RECEIVER][UNCAST-UDP] {}:{} -> {}:{} | {}".format(ip.src, int(udp.sport), ip.dst, int(udp.dport), text))
294
295     send_join()
296     next_join = time.time() + int(args.join_interval)
297
298     try:
299         while True:
300             now = time.time()
301             if now >= next_join:
302                 send_join()
303                 next_join = now
304             sniff(iface=iface,
305             except KeyboardInterrupt:
306                 print("\n[RECEIVER] Ctrl+C detected, exiting.")
307             try:
308                 send_leave()
309             except Exception as e:
310                 print("[RECEIVER] Error: {}".format(e))
311
312 if __name__ == "__main__":
313     main()
314
315
316
317

```

```

[RECEIVER] <<< Received unicast PONG from sender 10.0.1.10
[RECEIVER][MCAST-UDP] 10.0.1.10:5000 -> 239.1.1.1:5000 | 'Hello multicast #14 from 10.0.1.10'
[RECEIVER] >>> Sent unicast PING to sender 10.0.1.10
[RECEIVER] <<< Received unicast PONG from sender 10.0.1.10
[RECEIVER][MCAST-UDP] 10.0.1.10:5000 -> 239.1.1.1:5000 | 'Hello multicast #15 from 10.0.1.10'
[RECEIVER] >>> Sent unicast PING to sender 10.0.1.10
[RECEIVER] <<< Received unicast PONG from sender 10.0.1.10
[RECEIVER][MCAST-UDP] 10.0.1.10:5000 -> 239.1.1.1:5000 | 'Hello multicast #16 from 10.0.1.10'
[RECEIVER] >>> Sent unicast PING to sender 10.0.1.10
[RECEIVER] <<< Received unicast PONG from sender 10.0.1.10
[RECEIVER][MCAST-UDP] 10.0.1.10:5000 -> 239.1.1.1:5000 | 'Hello multicast #17 from 10.0.1.10'
[RECEIVER] >>> Sent unicast PING to sender 10.0.1.10
[RECEIVER] <<< Received unicast PONG from sender 10.0.1.10
[RECEIVER][MCAST-UDP] 10.0.1.10:5000 -> 239.1.1.1:5000 | 'Hello multicast #18 from 10.0.1.10'
[RECEIVER] >>> Sent unicast PING to sender 10.0.1.10
[RECEIVER] <<< Received unicast PONG from sender 10.0.1.10

```

شکل ۲: فاز اول – نسخه‌ی Scapy – اجرای Receiver 1 روی PC2: ارسال IGMP JOIN، دریافت پیام چندپخشی و ارسال PING.

در ادامه‌ی گزارش، تمرکز اصلی روی فاز دوم است: طراحی، پیاده‌سازی، بسته‌بندی داده‌های گرافیکی، همگام‌سازی بدون سرور، و تحلیل شبکه با Wireshark و Torch.

```

C:\Users\PCLOPREKIN\PC\Desktop> receiver.py ...
291         print("[RECEIVER] <<< Received unicast PONG from sender {}".format(ip.src))
292     else:
293         print("[RECEIVER][UCAST-UDP] {}:{} -> {}:{}".format(ip.src, int(udp.port), ip.dst, int(udp.dport), text))
294
295     send_join()
296     next_join = time.time() + int(args.join_interval)
297
298     try:
299         while True:
300             now = time.time()
301             if now >= next_join:
302                 send_join()
303                 next_join = now + int(args.join_interval)
304             sniff(iface=iface, filter=bpf, prn=handle,
305                 timeout=args.sniff_timeout)
306     except KeyboardInterrupt:
307         print("\n[RECEIVER] Ctrl+C detected.")
308         try:
309             send_leave()
310         except Exception as e:
311             print("[RECEIVER] LEAVE failed (ignored): {}".format(e))
312
313 if __name__ == "__main__":
314     main()
315
316
317
C:\Windows\py.exe
[RECEIVER] <<< Received unicast PONG from sender 10.0.1.10
[RECEIVER][MCAST-UDP] 10.0.1.10:5000 -> 239.1.1.1:5000 Hello multicast #19 fr
on 10.0.1.10
[RECEIVER] >>> Sent unicast PING to sender 10.0.1.10
[RECEIVER] <<< Received unicast PONG from sender 10.0.1.10
[RECEIVER][MCAST-UDP] 10.0.1.10:5000 -> 239.1.1.1:5000 Hello multicast #20 fr
on 10.0.1.10
[RECEIVER] >>> Sent unicast PING to sender 10.0.1.10
[RECEIVER] <<< Received unicast PONG from sender 10.0.1.10
[RECEIVER][MCAST-UDP] 10.0.1.10:5000 -> 239.1.1.1:5000 Hello multicast #21 fr
on 10.0.1.10
[RECEIVER] >>> Sent unicast PING to sender 10.0.1.10
[RECEIVER] <<< Received unicast PONG from sender 10.0.1.10
[RECEIVER][MCAST-UDP] 10.0.1.10:5000 -> 239.1.1.1:5000 Hello multicast #22 fr
on 10.0.1.10
[RECEIVER] >>> Sent unicast PING to sender 10.0.1.10
[RECEIVER] <<< Received unicast PONG from sender 10.0.1.10
[RECEIVER][MCAST-UDP] 10.0.1.10:5000 -> 239.1.1.1:5000 Hello multicast #23 fr
on 10.0.1.10
[RECEIVER] >>> Sent unicast PING to sender 10.0.1.10
[RECEIVER] <<< Received unicast PONG from sender 10.0.1.10

```

شکل ۳: فاز اول – نسخه‌ی Scapy – اجرای Receiver 2 روی PC3: دریافت همزمان با Receiver 1 و تبادل PING/PONG.

۲-۲ خلاصه نیازمندی‌های فاز دوم و مسیر تحقق آن‌ها

صورت‌مسئله‌ی فاز دوم چهار ویژگی فنی اصلی برای برنامه مشخص کرده است. در این گزارش، دقیقاً همان چهار بند را به عنوان ستون‌های طراحی در نظر می‌گیریم و نشان می‌دهیم چگونه در کد پیاده‌سازی شده‌اند.

۲-۲-۱ UDP + Multicast با پارامترهای قابل تنظیم

در برنامه‌ی پیاده‌سازی‌شده:

- آدرس گروه (group)، پورت (port) و TTL همگی از طریق CLI args قابل تنظیم هستند.
- تنظیم TTL به صورت صریح انجام می‌شود تا ارسال از یک شبکه به شبکه‌ی دیگر ممکن باشد (در توپولوژی ما حداقل $TTL \geq 2$ لازم است و مقدار 16 انتخاب شده است).
- برای جلوگیری از انتخاب اشتباه کارت شبکه (به دلیل وجود VMnet‌های متعدد)، برنامه یک پارامتر مهم به نام `ifaddr` دارد که مشخص می‌کند عضویت و ارسال چندپخش روی کدام Interface IP انجام شود.

۲-۲-۲ ارسال رویدادهای Canvas

هر عملیات گرافیکی (خط آزاد، قلم‌مو، پاک‌کن، خط مستقیم، مستطیل، بیضی، متن، تغییر پس‌زمینه، پاک‌کردن کل بوم، Undo) به یک پیام ساخت‌یافته تبدیل شده و در قالب یک بسته‌ی UDP ارسال می‌شود.

۲-۲-۳ دریافت و نمایش رویدادها

هر برنامه همزمان:

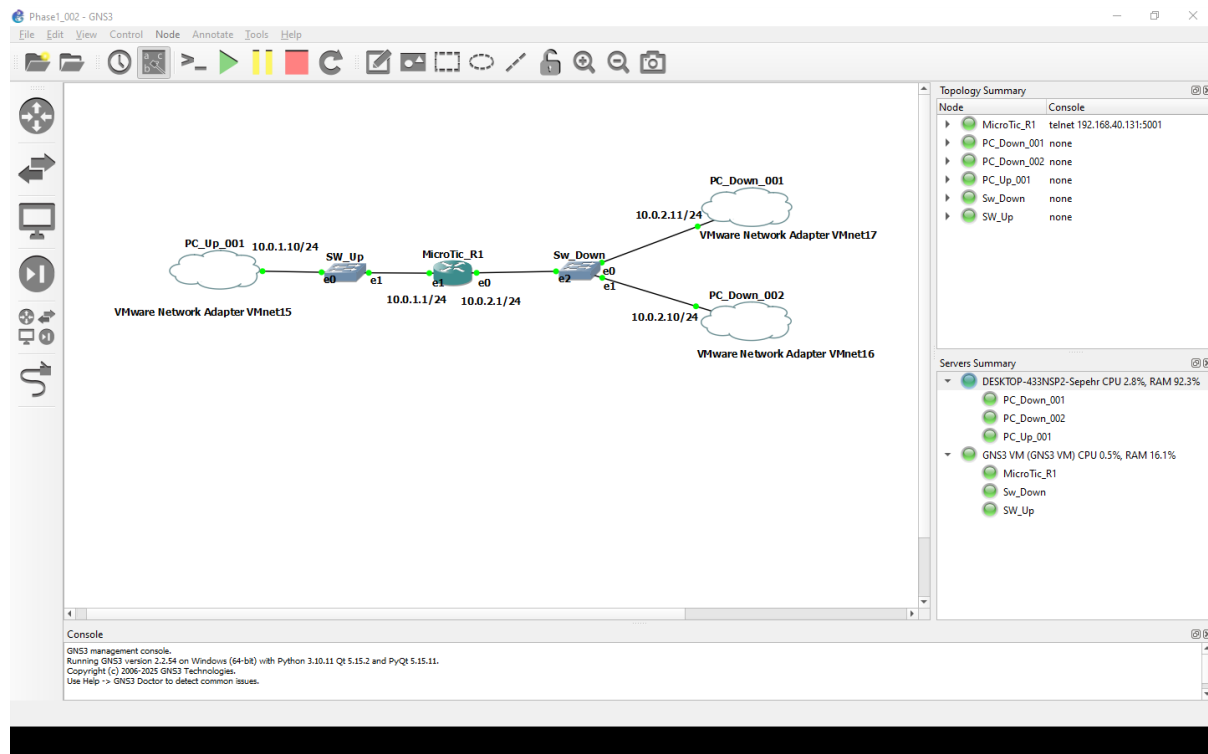
- در گروه Multicast عضو می شود و بسته ها را دریافت می کند؛
- داده را JSON parse می کند؛
- بر اساس type پیام، روی Canvas رسم می کند.

۲-۲-۴ Canvas مشترک و همگام سازی بدون سرور

- هیچ سرور مرکزی وجود ندارد.
- کشف همتایان (Peer Discovery) و همگام سازی صرفاً با پیام های hello/bye و خود ترافیک چندپخش انجام می شود.
- برای پایداری در سناریوی IGMP Proxy (که ماهیتاً عمدتاً مسیر Upstream->Downstream را پوشش می دهد)، یک مکانیزم **کمکی** اضافه شده است که در عین حفظ ارسال Multicast، در صورت نیاز پیام را به همتاهای شناخته شده **Unicast** نیز می کند تا ارتباط Downstream->Upstream هم عملی شود. (شرح کامل در بخش های بعد).

۲-۳ توپولوژی فیزیکی/منطقی و حالت های عملیاتی IGMP Proxy در برابر PIM

توپولوژی فاز دوم دقیقاً همان توپولوژی فاز اول است (یک روتر مرکزی MikroTik، یک سگمنت Upstream با یک میزبان، و یک سگمنت Downstream با دو گیرنده). برای جلوگیری از تکرار، تنها تصویر نهایی توپولوژی آورده می شود و جزئیات آدرس دهی و تست Unicast همان موارد فاز اول هستند.



شکل ۴: توپولوژی مشترک فاز اول و دوم: PC1 در Upstream و PC2/PC3 در Downstream با روتر مرکزی MikroTik.

۲-۳-۱: اجرای پروژه مطابق صورت مسئله با IGMP Proxy

در حالت استاندارد پروژه، روتر نقش IGMP Proxy دارد. در این حالت:

- عضویت‌های IGMP از سگمنت Downstream جمع‌آوری می‌شوند؛
- ترافیک چندپخش از Upstream به Downstream فوروارد می‌شود.

چالش مهم:

IGMP Proxy ذاتاً برای سناریوهای «یک منبع در یک سمت و چند گیرنده در سمت دیگر» طراحی شده است و طبق مستندات MikroTik، این روش بیش از یک **upstream** را پشتیبانی نمی‌کند و برای توپولوژی‌های پیچیده مناسب نیست^۱. در نتیجه، در بسیاری از پیاده‌سازی‌ها مسیر برگشت چندپخش از Downstream به Upstream یک مسیر استاندارد و پایدار ندارد. چون فاز دوم نیاز دارد همه کاربران **Up** و **Down** خطوط همدیگر را ببینند، اگر فقط به Multicast تکیه کنیم ممکن است «طراحی از Downstream به Upstream» دیده نشود.

برای حل این مسئله، در کد فاز دوم یک گزینه‌ی کمکی با نام **peer-unicast** – اضافه شده است:

^۱[help.mikrotik.com – IGMP Proxy \(RouterOS\)](https://help.mikrotik.com – IGMP Proxy (RouterOS))

- برنامه همچنان همیشه **Multicast** ارسال می کند (مطابق نیازمندی).
- علاوه بر آن، پیام را به همتایان شناخته شده **Unicast** هم می فرستد تا از محدودیت IGMP Proxy عبور کند.
- برای جلوگیری از دوباره نقاشی (چون ممکن است یک همتا هم Multicast و هم Unicast را دریافت کند)، یک مکانیزم Deduplication بر اساس Message ID وجود دارد.

۲-۳-۲ حالت ۲: اجرای پیشرفته با مسیریابی چندپخش واقعی PIM-SM

- در حالت پیشرفته (قدم اضافه)، روتر به جای صرفاً IGMP Proxy در نقش یک مسیریاب چندپخش واقعی تنظیم شد (PIM-SM در RouterOS). در این حالت:
- تفاوتی بین Upstream و Downstream از نظر عبور ترافیک چندپخش وجود ندارد؛
 - برنامه می تواند بدون نیاز به peer-unicast، صرفاً با Multicast در هر دو جهت کار کند.

چرایی انتخاب PIM-SM:

برای اینکه هر دو سمت بتوانند هم **Sender** باشند هم **Receiver** (یعنی عملاً «رفت و برگشت multicast بین دو شبکه»)، راه درست در RouterOS v7 این است که IGMP Proxy کنار گذاشته شود و **Routing Multicast** با PIM-SM فعال گردد. خود MikroTik مثال multicast Basic routing device single on را دقیقاً برای یک روتر با دو اینترفیس ارائه کرده است.^۲

۲-۳-۳ پیکربندی PIM-SM روی MikroTik (RouterOS) – دقیقاً مطابق تست پروژه

در این بخش، دقیقاً همان کانفیگی را که برای حالت PIM در پروژه استفاده شد، مرحله به مرحله می آوریم تا گزارش «قابل تکرار» باشد.

گام ۱: حذف IGMP Proxy

روی MikroTik:

```
/routing igmp-proxy interface print
/routing igmp-proxy interface remove [find]
```

نکته: همین که های IGMP Proxy interface حذف شوند، عملاً IGMP Proxy از کار می افتد و روتر دیگر به سبک Proxy فوروارده نمی کند.

²help.mikrotik.com – PIM-SM (RouterOS)

گام ۲: فعال کردن PIM-SM روی همان روتر با دو اینترنتیسی
(در توپولوژی پروژه، دو اینترنتیسی همان ether1 و ether2 هستند.)

```
/routing pimsm instance add name=pim1
/routing pimsm interface-template add interfaces=ether1,ether2 instance=pim1
```

طبق داکيومنت MikroTik، همین حداقل کانفیگ باعث می‌شود روتر پیام‌های IGMP Join کلاینت‌ها را بشنود و ترافیک multicast را بین سگمنت‌ها route کند.^۳

گام ۳ (پیشنهادی): اطمینان از Drop نشدن IGMP و UDP Multicast در فایروال
اگر روی RouterOS فایروال filter دارید (در بعضی کانفیگ‌های پیش‌فرض)، این دو rule کمک می‌کند:

```
/ip firewall filter add chain=forward action=accept protocol=igmp comment="Allow I
/ip firewall filter add chain=forward action=accept protocol=udp dst-address=224.0
```

گام ۴: تست و مانیتورینگ جدول‌های PIM
بعد از اجرای برنامه‌ها و Join شدن گیرنده‌ها:

```
/routing pimsm uib-g print
/routing pimsm uib-sg print
```

این دستورات طبق مستندات MikroTik گروه‌های فعال و (S,G)های دیده‌شده را نشان می‌دهند.^۴

نکته مهم درباره «دوطرفه»

اگر منظور از «دوطرفه» دقیقاً PIM-Bidir باشد: مستندات رسمی RouterOS برای این بخش عمدتاً PIM-SM را پوشش می‌دهد و برای نیاز پروژه (ارسال از هر دو سمت به یک گروه و دریافت در سمت مقابل) همین PIM-SM روی یک روتر کاملاً کافی و پایدار است.^۵

^۳help.mikrotik.com – PIM-SM (RouterOS)

^۴help.mikrotik.com – PIM-SM (RouterOS)

^۵help.mikrotik.com – PIM-SM (RouterOS)

در بخش تست‌ها، خروجی اجرای برنامه در هر دو حالت ارائه شده است (تصاویر ۹ و ۱۰).

۲-۴ معماری برنامه‌ی فاز دوم و تقسیم‌بندی منطقی کد

برای اینکه خواننده در یک کد بزرگ گم نشود، ابتدا ساختار کلی برنامه را به چند ماژول مفهومی تقسیم می‌کنیم. فایل اصلی برنامه `mcast_canvas_paint.py` از نظر منطقی شامل این بلوک‌هاست:

- **لایه‌ی شبکه (Data Plane) بر پایه‌ی socket:** کلاس `MulticastTransport` که وظیفه‌ی `join` کردن به گروه، تنظیم `TTL`، ارسال/دریافت `UDP` و ثبت آمار شبکه را بر عهده دارد.

- **لایه‌ی اختیاری Scapy برای IGMP:** کلاس `ScapyIGMPKeepalive` که در صورت فعال شدن – `igmp`، پیام‌های `IGMPv2 JOIN` را به صورت دوره‌ای ارسال و هنگام خروج `LEAVE` می‌فرستد (فقط برای رویت‌پذیری و پایدار نگه داشتن عضویت‌ها؛ داده‌ی اصلی همچنان با `socket` جابه‌جا می‌شود).

- **هسته‌ی برنامه‌ی Canvas:** کلاس `MulticastPaintApp` که:

- رابط کاربری `Tkinter` را می‌سازد مشابه `Paint`
- رویدادهای ماوس را به پیام شبکه تبدیل می‌کند،
- پیام‌های دریافتی را روی بوم رسم می‌کند،
- وضعیت هم‌تاها، آمار و `Undo/Redo` را مدیریت می‌کند.

- **همروندی (Threading) برای جلوگیری از فریز شدن UI:**

- یک `Thread` دریافت شبکه دارد که `recvfrom` می‌کند،
- داده را داخل یک `Queue` می‌گذارد،
- و در `Main thread` با `after()` به صورت دوره‌ای صف پردازش می‌شود.

چرایی این معماری:

`Tkinter` از نظر `Thread-safety` حساس است و انجام عملیات رسم روی `Canvas` از `Thread` دیگر ممکن است باعث کرش یا رفتار غیرقابل پیش‌بینی شود. بنابراین طراحی `Thread` شبکه + صف + پردازش در `Main thread` دو مزیت کلیدی دارد:

- **عدم بلاک شدن UI** دریافت شبکه می‌تواند بلاک شود، ولی UI آزاد می‌ماند.

• **رندر امن:** تمام عملیات Canvas در همان Thread اصلی انجام می‌شود.

۵-۲ پیاده‌سازی لایه شبکه با Socket و پیکربندی Multicast

۵-۲-۱ تنظیم و عضویت در گروه چندپخش (Join) روی Interface مشخص

یکی از مشکلات رایج در محیط‌های مجازی (به خصوص در ویندوز + VMware VMnet) این است که اگر IP_ADD_MEMBERSHIP با INADDR_ANY انجام شود، سیستم عامل ممکن است عضویت را روی کارت شبکه‌ی اشتباه اعمال کند. به همین دلیل در این برنامه، **عضویت با IP دقیق کارت شبکه** انجام می‌شود.

قطعه کد کلیدی در `MulticastTransport.__init__`:

```
self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
self.sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
try:
    self.sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEPORT, 1)
except Exception:
    pass

# Bind for receiving
self.sock.bind(("", self.port))

group_bin = socket.inet_aton(self.group)
ifaddr_bin = socket.inet_aton(self.ifaddr)

# Join group on specific interface
mreq = struct.pack("4s4s", group_bin, ifaddr_bin)
self.sock.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)
```

نکات مهم این بخش:

• `SO_REUSEPORT/SO_REUSEADDR` برای اجرای چند برنامه یا ری‌استارت سریع مفید است.

• `bind("", port)` باعث می‌شود سوکت روی همه‌ی آدرس‌ها روی آن پورت شنود کند.

• IP_ADD_MEMBERSHIP با (group, ifaddr) باعث می‌شود عضویت دقیقاً روی همان کارت انجام شود.

۲-۵-۲ تنظیم اینترفیس خروجی و TTL

پس از join، برنامه برای ارسال درست باید مشخص کند:

• بسته‌های چندپخشی از کدام interface خارج شوند؛

• مقدار TTL چند باشد (برای عبور از روتر، $TTL \geq 2$).

کد مربوطه:

```
# Outgoing interface and TTL
self.sock.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_IF, ifaddr_bin)
self.sock.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_TTL, struct.pack("B",
```

در توپولوژی پروژه، مقدار پیش‌فرض $TTL=16$ انتخاب شده است تا:

• عبور از روتر قطعی باشد،

• و در تحلیل Wireshark نیز با کاهش TTL پس از عبور از روتر بتوان مسیر را تشخیص داد

مثلاً ۱۶ در مبدأ، ۱۵ پس از یک Hop

۲-۵-۳ کنترل Loopback و بهینه‌سازی بافرها

برای کاهش از دست‌رفتن بسته‌ها هنگام کشیدن سریع خطوط، (Burst) بافرها بزرگ‌تر

شده‌اند:

```
self.sock.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 2 * 1024 * 1024)
self.sock.setsockopt(socket.SOL_SOCKET, socket.SO_SNDBUF, 2 * 1024 * 1024)
```

همچنین IP_MULTICAST_LOOP کنترل می‌کند آیا کاربر بسته‌ی Multicast خودش را هم

ببیند یا خیر. در حالت عادی این گزینه خاموش است تا «دوباره‌نقاشی» رخ ندهد. (در صورت نیاز

برای Debug می‌توان آن را روشن کرد).

۲-۵-۴ چرخه‌ی ارسال/دریافت در سطح بایت (Encoding/Decoding) و ثبت شمارنده‌ها

در این پروژه داده‌ها با حداقل سربرار و به صورت قابل خواندن در Wireshark ارسال می‌شوند:

- تبدیل ساختار پیام به رشته‌ی JSON با `separators=(",", ":")` تا فضای خالی حذف شود.
- **Encoding** به صورت UTF-8، سپس ارسال با `.sendto`.
- روی گیرنده: دریافت با `recvfrom`، `decode("utf-8", errors="ignore")` و سپس `json.loads`.
- کد ارسال (multicast/unicast) و شماره‌ها:

```
data = json.dumps(obj, separators=(",", ":")).encode("utf-8", errors="ignore")
self.sock.sendto(data, addr)
# counters: packets + bytes
```

کد دریافت:

```
data, addr = self.sock.recvfrom(65535)
obj = json.loads(data.decode("utf-8", errors="ignore"))
```

نتیجه‌ی این طراحی: Payload در Wireshark به راحتی به صورت Text قابل مشاهده است و برای «تحلیل پروتکلی» نیاز به Decoder اختصاصی نداریم.

۲-۶ قالب پیام‌ها (Message Schema) و پروتکل سطح کاربرد

هسته‌ی فاز دوم این است که «رویداد گرافیکی» به یک «پیام شبکه» تبدیل شود. برای ساده‌بودن و قابل تحلیل بودن در Wireshark، قالب پیام‌ها JSON انتخاب شده است.

۲-۶-۱ فیلدهای عمومی مشترک بین همه پیام‌ها

تقریباً همه‌ی پیام‌ها این فیلدها را دارند:

- v: نسخه‌ی پیام (در این پروژه 1)
- type: نوع پیام (مثلاً hello, seg, stroke, shape, text, bg, ...)
- mid: شناسه‌ی یکتای پیام برای Deduplication
- id: شناسه‌ی یکتای کلاینت (برای تشخیص «پیام خودم» و مدیریت هم‌تاها)
- user: نام کاربری نمایشی

• ts: زمان ارسال به میلی‌ثانیه

در برخی نسخه‌های تست/دیب‌اگ، یک شمارنده‌ی محلی نیز برای خوانایی در Wireshark اضافه شده است:

• lc: یک Local Counter صرفاً برای تحلیل ترتیب پیام‌ها (در منطق برنامه نقشی ندارد و حذف/وجود آن خللی ایجاد نمی‌کند).

ساخت mid و id با uuid انجام می‌شود:

```
self.client_id = str(uuid.uuid4())
...
def _new_mid(self) -> str:
    return uuid.uuid4().hex
```

۲-۶-۲ نرمال‌سازی مختصات برای مستقل بودن از اندازه‌ی Canvas

چون ممکن است اندازه‌ی پنجره‌ی کاربران متفاوت باشد، مختصات به جای پیکسل خام، به شکل نسبت‌های نرمال‌شده ارسال می‌شود:

$$x_n = \frac{x}{W}, \quad y_n = \frac{y}{H}$$

کد مربوطه:

```
def _norm(self, x: int, y: int) -> List[float]:
    w, h = self._canvas_size()
    return [round(x / float(w), 6), round(y / float(h), 6)]

def _denorm(self, p: List[float]) -> Tuple[int, int]:
    w, h = self._canvas_size()
    x = int(p[0] * w)
    y = int(p[1] * h)
    return x, y
```

نتیجه: اگر یک کاربر در گوشه‌ی بالا-چپ یک شکل بکشد، کاربران دیگر نیز همان موقعیت نسبی را می‌بینند، حتی اگر پنجره‌شان بزرگ‌تر یا کوچک‌تر باشد.

۲-۶-۳ جدول خلاصه‌ی پیام‌ها، زمان تولید و اندازه‌ی تقریبی (Packet Budget)

برای «تحلیل شبکه» و نیز کنترل، MTU دانستن حدود اندازه پیام‌ها مهم است. در پروژه:

• UTF-8 JSON Encoding:

• فشرده‌سازی: ندارد برای شفافیت در Wireshark

• محدودیت بسته: max_packet_bytes=1300 برای پیام‌های بزرگ مثل stroke

نوع پیام	چه زمانی ارسال می‌شود؟	مهم‌ترین فیلدها	Payload معمولی
hello	هر presence_interval ثانیه برای اعلام حضور	id,user,port,ts,mid	160–170 B
bye	هنگام خروج برنامه	id,user,ts,mid	130–160 B
seg	در حین حرکت ماوس برای Pencil/Brush/Eraser	stroke, seq, p0, p1, color, width, eraser	200–320 B
stroke	هنگام رها کردن ماوس (نسخه نهایی Stroke)	stroke, pts, color, width, smooth, op	400–1300 B
shape	هنگام رها کردن ماوس در ابزارهای Line/Rect/Oval	shape, p0, p1, outline, fill, fill_color, width, op	280–450 B
text	بعد از تایید پنجره متن	pos, text, color, size, op	250–600 B
bg	ابزار Bucket برای تغییر پس‌زمینه	bg	120–180 B
clear	پاک کردن همگانی بوم	ts,user	120–180 B
undo	Undo همگانی یک عملیات نهایی	op	150–220 B

جدول ۱: خلاصه‌ی پیام‌ها و حدود اندازه‌ی Payload. اندازه‌ها وابسته به طول نام کاربری، تعداد نقاط، و فعال بودن فیلدهای دیباگ مثل lc هستند.

نکته MTU

در شبکه‌های اترنت معمولی، MTU=1500 است. اگر Payload را حدود 1300 B نگه داریم، حتی با سربراهای IP(20) و UDP(8) و Ethernet(14) همچنان با فاصله‌ی امن زیر 1500 می‌مانیم و احتمال Fragmentation عملاً حذف می‌شود.

۷-۲ بسته‌بندی رویدادها: از رویداد UI تا Packet و سپس بازسازی روی گیرنده

این بخش یک نگاه «از بالا به پایین» می‌دهد که دقیقاً هر رویداد گرافیکی چگونه به بسته‌ی شبکه تبدیل می‌شود و در سمت مقابل چگونه بازسازی می‌گردد.

۱-۷-۲ مسیر یک پیام از UI تا شبکه (Send Path)

برای هر رویداد (مثلاً رسم یک بیضی):

۱. کاربر با ماوس رویداد تولید می‌کند (فشار، حرکت، رهاسازی).

۲. برنامه بر اساس ابزار فعال، یک dict پیام می‌سازد و فیلدهای عمومی مثل mid,id,ts را اضافه می‌کند.

۳. پیام با json.dumps(...) به JSON فشرده تبدیل می‌شود (بدون فاصله).

۴. JSON به UTF-8 تبدیل می‌شود و با sendto به (group,port) ارسال می‌گردد.

۲-۷-۲ مسیر یک پیام از شبکه تا UI (Receive & Render Path)

۱. Thread شبکه با recvfrom بسته را دریافت می‌کند.

۲. Payload به UTF-8 decode و سپس json.loads می‌شود.

۳. پیام همراه آدرس فرستنده وارد Queue می‌شود.

۴. در Main، thread تابع Queue-pump پیام‌ها را یکی یکی برمی‌دارد.

۵. ابتدا Dedup انجام می‌شود (براساس mid).

۶. سپس بر اساس type، تابع مربوطه اجرا و روی Canvas رسم می‌شود.

دلیل صف:

اگر بخواهیم در Thread شبکه مستقیماً روی Canvas رسم کنیم، در بسیاری از سیستم‌ها Tkinter ناسازگار می‌شود. صف باعث می‌شود هیچ دستور گرافیکی خارج از thread Main اجرا نشود.

۲-۸ بسته‌بندی رویدادهای ترسیم و الگوریتم Real-Time Drawing

این بخش مهم‌ترین قسمت فاز دوم است: «چگونه عمل رسم به بسته‌های شبکه تبدیل می‌شود» و «چگونه روی گیرنده دوباره بازسازی می‌شود». در برنامه، دو لایه برای ترسیم در نظر گرفته شده است:

- **لایه‌ی سریع (Segment Streaming):** هنگام کشیدن خط آزاد با ماوس، به ازای حرکت‌های کوچک، بسته‌های کوچک seg ارسال می‌شود تا گیرنده‌ها در حین کشیدن خطوط را ببینند.
- **لایه‌ی نهایی (Stroke Finalization):** هنگام رها کردن دکمه‌ی ماوس، یک بسته‌ی stroke شامل مجموعه نقاط ارسال می‌شود تا گیرنده‌ها بتوانند خطوط را یکپارچه و نرم ترسیم کنند و از تعداد زیاد قطعه‌خط‌ها جلوگیری شود.

۲-۸-۱ ارسال seg در زمان حرکت ماوس برای Pencil/Brush/Eraser

وقتی ابزار روی یکی از حالت‌های آزاد باشد: Pencil / Brush / Eraser در رویداد <B1-Motion> کد زیر اجرا می‌شود:

```
if tool in (TOOL_PENCIL, TOOL_BRUSH, TOOL_ERASER):
    ...
    dist = math.hypot(x1 - x0, y1 - y0)
    if dist < float(self.cfg.min_step_px):
        return

seg_msg = {
    "v": 1,
    "type": "seg",
    "mid": self._new_mid(),
    "id": self.client_id,
    "user": self.cfg.username,
    "ts": now_ms(),
    "stroke": self.local_stroke_id,
    "seq": self.local_seq,
    "p0": self._norm(x0, y0),
```

```
"p1": self._norm(x1, y1),
"color": ...,
"width": int(width),
"eraser": bool(eraser),
}
self._broadcast_obj(seg_msg)
```

نکات کلیدی:

- min_step_px (پیش فرض ۲ پیکسل) باعث می شود برای لرزش های خیلی ریز بسته تولید نشود و ترافیک کنترل گردد.

- هر seg شامل دو نقطه p0,p1 است: یعنی یک قطعه خط.

- stroke یک UUID برای کل حرکت قلم است تا گیرنده بداند قطعات متعلق به یک Stroke هستند.

- seq شماره ی ترتیبی قطعه هاست (برای تحلیل و در صورت نیاز مدیریت ترتیب).

- برای پاک کن، رنگ عملاً رنگ پس زمینه است و eraser=true نیز ست می شود تا گیرنده دقیقاً همان رفتار را اعمال کند.

رفتار روی گیرنده: در تابع handle_seg، این قطعه ها فوراً روی Canvas رسم می شوند و در یک ساختار remote_temp_items[stroke_id] نگهداری می شوند تا بعداً بتوان آن ها را با نسخه ی نهایی جایگزین کرد:

```
it = self._draw_segment(x0, y0, x1, y1, color, width)
self.remote_temp_items.setdefault(stroke_id, []).append(it)
```

۲-۸-۲) ارسال stroke در لحظه ی رها کردن ماوس

وقتی کاربر دکمه ی ماوس را رها می کند، برنامه علاوه بر قطعه خط های قبلی، یک پیام نهایی می فرستد که شامل لیست نقاط است:

```
stroke_msg = {
    "v": 1,
    "type": "stroke",
    "mid": self._new_mid(),
    "id": self.client_id,
    "user": self.cfg.username,
    "ts": now_ms(),
    "op": op_id,
    "stroke": self.local_stroke_id,
    "pts": pts,          # list of [x_n,y_n]
    "color": color,
    "width": int(width),
    "eraser": bool(eraser),
    "smooth": bool(self.smooth_var.get()),
}

self._broadcast_obj(stroke_msg)
```

دو مکانیزم بهینه‌سازی مهم اینجا وجود دارد:

- **Downsampling:** اگر تعداد نقاط زیاد شود، تابع `downsample_points` تعداد را تا `max_stroke_points` (پیش‌فرض ۸۰) کاهش می‌دهد تا بسته خیلی بزرگ نشود.
- **کنترل اندازه بسته:** اگر با وجود کاهش نقاط، اندازه‌ی JSON از `max_packet_bytes` (پیش‌فرض ۱۳۰۰ بایت) بزرگ‌تر شود، دوباره نقاط کمتر می‌شود تا زیر این حد بیاید.

کد مربوط به کنترل اندازه‌ی بسته:

```
raw = json.dumps(stroke_msg, separators=(",", ":")).encode("utf-8", errors="ignore")
if len(raw) > self.cfg.max_packet_bytes and len(pts) > 10:
    shrink = len(pts)
    while len(raw) > self.cfg.max_packet_bytes and shrink > 10:
        shrink = max(10, shrink // 2)
```

```
pts2 = downsample_points(pts, shrink)
stroke_msg["pts"] = pts2
raw = json.dumps(stroke_msg, separators=(",", ":")).encode("utf-8", errors="ignore")
```

رفتار روی گیرنده: وقتی پیام stroke رسید:

- تمام قطعه خط‌های موقتی مرتبط با آن stroke_id حذف می‌شوند؛
- یک خط نرم (smooth polyline) با create_line و splinesteps کشیده می‌شود؛
- و در ops ثبت می‌شود تا Undo ممکن باشد.

```
# remove temp segments for this stroke if any
old = self.remote_temp_items.pop(stroke_id, [])
for it in old:
    self.canvas.delete(it)

poly_id = self._draw_polyline(pts_px, color, width, smooth)
self.remote_final[stroke_id] = now_ms()
return [poly_id]
```

این طراحی باعث می‌شود:

- کاربرها در حین کشیدن خط، بازخورد فوری ببینند.
- در پایان، به جای صدها آیتم کوچک Canvas، یک آیتم تمیز و نرم ثبت شود.
- حتی اگر چند seg در شبکه Drop شود، پیام نهایی stroke «وضعیت پایانی» را تا حد زیادی تصحیح می‌کند.

۹-۲ پیام‌های مربوط به ابزارهای شبیه Paint و نحوه‌ی ارسال هر کدام

این برنامه علاوه بر «خط آزاد»، قابلیت‌های بیشتری شبیه Windows Paint دارد. در این بخش، برای هر ابزار توضیح می‌دهیم چه پیام شبکه‌ای تولید می‌شود و گیرنده چگونه آن را رسم می‌کند.

۹-۲-۱) ابزارهای Shape: خط مستقیم، مستطیل، بیضی

در ابزارهای Line/Rectangle/Oval:

• در زمان حرکت ماوس، فقط یک **Preview محلی** روی بوم رسم می‌شود (روی شبکه ارسال نمی‌شود) تا کاربر قبل از نهایی کردن شکل آن را ببیند.

• با رها کردن ماوس، یک پیام shape ارسال می‌شود.

قطعه کد پیام shape:

```
shape_msg = {
    "v": 1,
    "type": "shape",
    "mid": self._new_mid(),
    "id": self.client_id,
    "user": self.cfg.username,
    "ts": now_ms(),
    "op": op_id,
    "shape": shape,          # "line" / "rect" / "oval"
    "p0": self._norm(x0, y0),
    "p1": self._norm(x1, y1),
    "outline": outline,
    "width": int(width),
    "fill": bool(fill is not None),
    "fill_color": fill if fill else "",
}
self._broadcast_obj(shape_msg)
```

توضیح فیلدها:

• shape: نوع شکل.

• p0,p1: دو گوشه‌ی شکل (یا ابتدا/انتهای برای خط).

• outline: رنگ دور خط.

• fill: اگر گزینه‌ی Fill shapes فعال باشد true.

• fill_color: رنگ پرکردن.

نکته‌ی مهم درباره نگاشت رنگ‌ها در Shapeها:

برای نزدیک شدن به تجربه Paint و همچنین مطابق سناریوی تست Wireshark این گزارش:

• Outline از Secondary گرفته می‌شود.

• Fill (در صورت فعال بودن) از Primary گرفته می‌شود.

بنابراین «بیضی پرشده با دورخط ثانویه» دقیقاً همان چیزی است که در بسته‌ی shape با فیلدهای outline و fill_color دیده می‌شود.

رفتار روی گیرنده: گیرنده با create_line/create_oval/create_rectangle همان شکل را رسم

می‌کند:

```
item = self._draw_shape_item(shape, x0, y0, x1, y1, outline, width,
fill_color if fill else None)
```

۲-۹-۲) ابزار Text

وقتی ابزار Text انتخاب شود:

• با کلیک روی بوم یک پنجره‌ی کوچک Toplevel باز می‌شود؛

• متن گرفته می‌شود؛

• یک پیام text ارسال می‌شود.

پیام text:

```
msg = {
    "v": 1,
    "type": "text",
    "mid": self._new_mid(),
    "id": self.client_id,
    "user": self.cfg.username,
    "ts": now_ms(),
```

```
"op": op_id,
"pos": self._norm(x, y),
"text": text,
"color": color,
"size": size,
}
self._broadcast_obj(msg)
```

۲-۹-۳) ابزار Bucket: تغییر پس زمینه (BG Fill)

این ابزار یک «پرکردن ساده‌ی پس‌زمینه» است (برای ساده‌سازی، کل پس‌زمینه‌ی بوم را تغییر می‌دهد). پیام bg:

```
msg = {
  "v": 1,
  "type": "bg",
  "mid": self._new_mid(),
  "id": self.client_id,
  "user": self.cfg.username,
  "ts": now_ms(),
  "bg": bg,
}
```

روی گیرنده:

```
self.cfg.bg = bg
self.canvas.configure(background=bg)
```

۲-۹-۴) Local New و Broadcast Clear

• **Local New** فقط بوم همان کاربر را پاک می‌کند و روی شبکه چیزی نمی‌فرستد (برای اینکه یک نفر ناخواسته کار دیگران را پاک نکند).

• **(Broadcast) Clear** برای همه کاربران پاک می‌کند و پیام clear می‌فرستد.

پیام clear:

```
msg = {
    "v": 1,
    "type": "clear",
    "mid": self._new_mid(),
    "id": self.client_id,
    "user": self.cfg.username,
    "ts": now_ms(),
}
self._broadcast_obj(msg)
```

۲-۹-۵ (Broadcast) Undo/Redo و مفهوم Operation ID

یکی از بخش‌های مهم برای شبیه شدن به Paint پشتیبانی از Undo/Redo است. برای اینکه Undo بین همه همگام شود، برنامه مفهوم Operation ID را دارد:

- هر رسم نهایی (یک stroke یا shape یا text) دارای op است.
- در ساختار ops[op_id]، آیتم‌های رسم‌شده‌ی مربوط به آن op ذخیره می‌شود.
- وقتی Undo زده می‌شود، پیام undo شامل op روی شبکه پخش می‌شود.

پیام undo:

```
msg = {
    "v": 1,
    "type": "undo",
    "mid": self._new_mid(),
    "id": self.client_id,
    "user": self.cfg.username,
    "ts": now_ms(),
}
```

```
"op": op_id,
}

self._broadcast_obj(msg)
```

روی گیرنده، آیتم‌های مربوط حذف می‌شوند و active=false می‌شود.

Redo: در برنامه، Redo با «ارسال دوباره‌ی همان پیام اصلی» انجام می‌شود (با یک mid جدید تا dedup آن را حذف نکند). این روش باعث می‌شود حالت Redo نیز بدون سرور و به شکل همگام قابل انجام باشد.

۲-۱۰ کشف همتاها (Peer Discovery) پیام‌های hello/bye و حل محدودیت Proxy IGMP با Assist Unicast

۲-۱۰-۱ hello/bye و نمایش لیست همتاها

برای اینکه برنامه بتواند بدون سرور، بفهمد چه کسانی روی شبکه فعال هستند:

- به صورت دوره‌ای پیام hello ارسال می‌شود؛
- هنگام خروج نیز پیام bye ارسال می‌شود؛
- هر برنامه با دریافت این پیام‌ها جدول peers را به‌روزرسانی کرده و لیست Peers را در UI نمایش می‌دهد.

پیام hello:

```
hello = {
    "v": 1,
    "type": "hello",
    "mid": self._new_mid(),
    "id": self.client_id,
    "user": self.cfg.username,
    "ts": now_ms(),
    "port": self.cfg.port,
}
```

به طور پیش فرض هر `presence_interval=5s` یک `hello` ارسال می شود و اگر یک همتا تا ۳۰ ثانیه دیده نشود از لیست حذف می گردد.

۲-۱۰-۲ چرا `peer-unicast` لازم شد؟

در حالت IGMP Proxy اگر:

- PC1 در Upstream ارسال کند $\Rightarrow$ روتر می تواند به Downstream بازپخش کند (طبق طراحی فاز اول).

- اما اگر PC2 یا PC3 در Downstream ارسال کند $\Rightarrow$ لزوماً روتر آن را به Upstream فوروارد نمی کند (چون نقش Proxy عموماً Up $\rightarrow$ Down است).

بنابراین برای «دوطرفه» کردن برنامه در حالت IGMP Proxy، مکانیزم کمکی اضافه شده است.

۲-۱۰-۳ مکانیزم `peer-unicast` چگونه کار می کند؟

تابع مرکزی ارسال در برنامه `_broadcast_obj` است:

```
def _broadcast_obj(self, obj):
    # Always send multicast
    self.transport.send_multicast(obj)

    if not self.cfg.peer_unicast:
        return

    # Unicast assist: send to known peers too
    for pid, info in list(self.peers.items()):
        ip = info.get("ip")
        port = info.get("port", self.cfg.port)
        if isinstance(ip, str) and ip and pid != self.client_id:
            self.transport.send_unicast(obj, ip, int(port))
```

توضیح:

- ارسال همیشه Multicast انجام می شود (شرط پروژه حفظ می شود).

- اگر peer-unicast – فعال باشد، همان پیام با همان mid به صورت Unicast هم برای همتاها ارسال می‌شود.
- همتاها از طریق hello و ترافیک دریافتی کشف می‌شوند (آدرس IP همان addr[0] در (recvfrom).

Deduplication برای جلوگیری از دوباره رسم کردن:

اگر در حالت PIM (یا شبکه‌ای که Multicast دوطرفه است) همزمان هم Multicast و هم Unicast برسد، باید یکی حذف شود. بنابراین هر پیام یک mid دارد و در ()_seen بررسی می‌شود.

```
def _seen(self, mid):
    if not isinstance(mid, str) or not mid:
        return False
    if mid in self._seen_set:
        return True
    self._seen_set.add(mid)
    self._seen_q.append(mid)
    ...
    return False
```

نکته پایداری:

برای جلوگیری از رشد بی‌نهایت حافظه، SEEN_MAX محدود است و deque(maxlen=...) باعث می‌شود قدیمی‌ها به صورت خودکار حذف شوند.

نتیجه: برنامه هم در حالت IGMP Proxy پایدار است (به کمک Unicast و هم در حالت PIM بدون مشکل اضافه کار می‌کند (چون Duplicate حذف می‌شود).

۲-۱۱ قابلیت‌های رابط کاربری شبیه Paint و مستندسازی آن‌ها با تصویر

در این بخش دقیقاً ویژگی‌هایی را که باعث شده برنامه «صرفاً یک نمایش خط ساده» نباشد و بیشتر شبیه Paint شود، مرور می‌کنیم. این ویژگی‌ها مستقیماً در کد و UI پیاده‌سازی شده‌اند.

۲-۱۱-۱ ساختار کلی UI

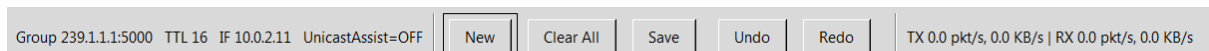
- **نوار بالا Ribbon:** نمایش اطلاعات شبکه، دکمه‌های New/Clear/Save/Undo/Redo و نمایش آمار ترافیک.

• **ستون چپ (Toolbox):** انتخاب ابزارها، گزینه‌ها (Smooth، Fill) و تنظیم اندازه قلم و فونت.

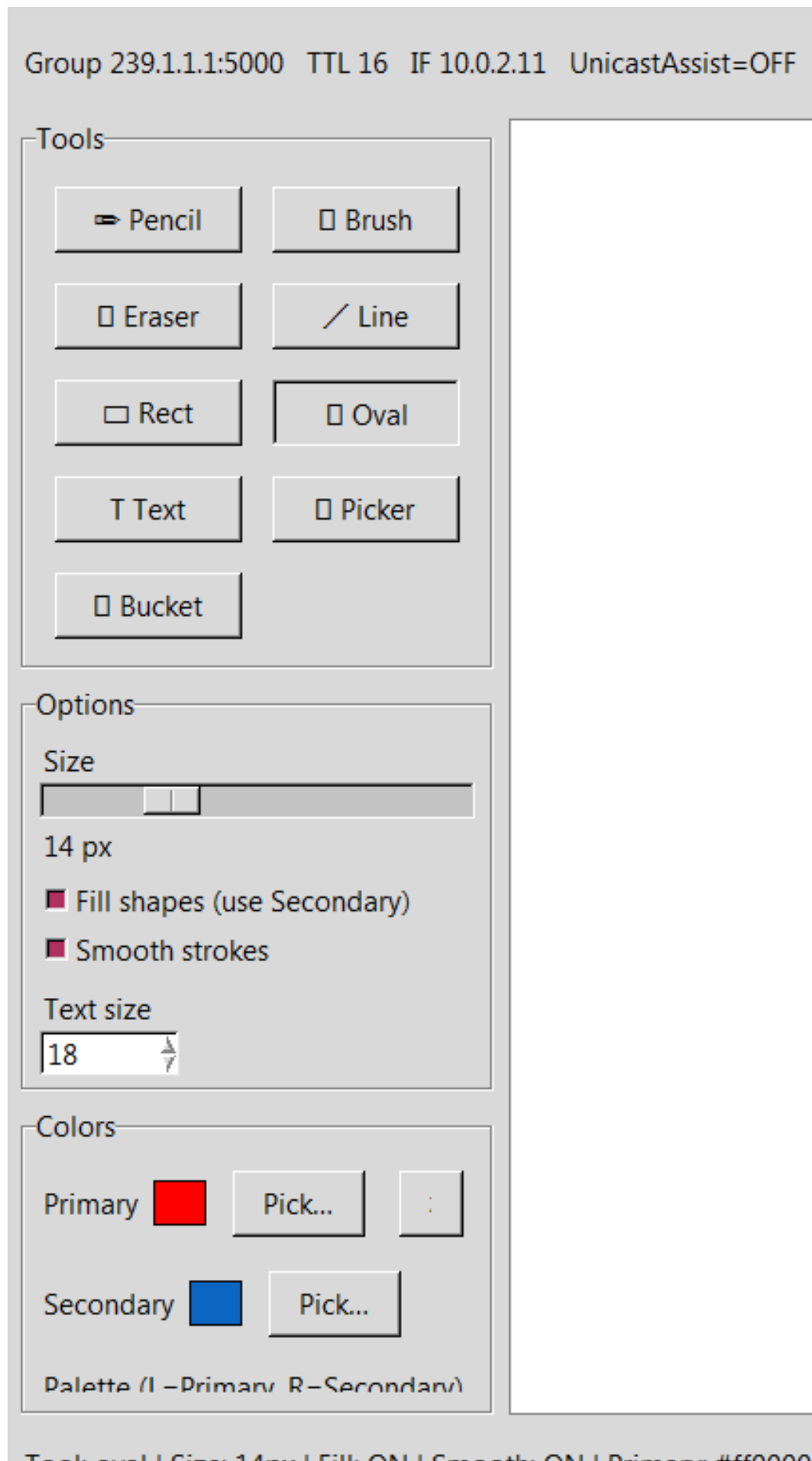
• **وسط:** Canvas اصلی برای ترسیم.

• **ستون راست:** Peers (همتاها) + آخرین پیام دریافت‌شده.

• **نوار وضعیت پایین:** نمایش ابزار فعال، اندازه، رنگ‌ها، مختصات ماوس، تعداد همتاها و نرخ ارسال/دریافت.



شکل ۵: نمای نوار بالایی برنامه: اطلاعات Group/Port/TTL/IF، دکمه‌های مدیریتی و بخش آمار ارسال/دریافت.



شکل ۶: نمای ستون ابزارها: (Toolbox) انتخاب ابزارها، گزینه‌ها، تنظیم اندازه و مدیریت رنگ‌های Primary/Secondary و پالت.

۲-۱۱-۲ ابزارها و رفتار دقیق هر کدام

- **Pencil**: خط باریک (عرض محدود شده تا شبیه مداد باشد).
- **Brush**: خط ضخیم تر (برای نقاشی سریع).
- **Eraser**: در واقع یک قلم با رنگ پس زمینه که `eraser=true` هم در پیام ثبت می شود تا گیرنده نیز دقیقاً همان رفتار را اعمال کند.
- **Line**: خط مستقیم با دو نقطه `p0,p1`.
- **Rectangle**: مستطیل با دو گوشه `p0,p1` و گزینه `Fill shapes`.
- **Oval**: بیضی با دو گوشه `p0,p1`.
- **Text**: درج متن در مختصات دلخواه با اندازه فونت قابل تنظیم.
- **Picker Color**: انتخاب رنگ از روی بوم (با خواندن رنگ آیتم زیر ماوس).
- **(BG) Bucket**: تغییر پس زمینه کل بوم.

۳-۱۱-۲ پالت رنگ و دو رنگ Primary/Secondary

در طراحی این برنامه:

- کلیک چپ روی رنگ ها $\Rightarrow$ رنگ **Primary**.
 - کلیک راست روی رنگ ها $\Rightarrow$ رنگ **Secondary**.
- در ابزارهای Shape (مطابق سناریوی تحلیلی Wireshark در این گزارش):

• **Secondary = Outline**

• **Primary = Fill** (اگر `Fill shapes` فعال باشد)

این نگاشت دقیقاً باعث می شود بتوانیم در یک تست، «بیضی پر شده با دورخط ثانویه» تولید کنیم و اثر آن را در پیام shape ببینیم.

۴-۱۱-۲ نمایش آمار ترافیک در UI

برنامه علاوه بر عملکرد اصلی، یک بخش آمار دارد که نرخ ارسال/دریافت را به صورت `pkt/s` و `KB/s` نمایش می دهد. این بخش برای تحلیل زنده ی «فشار شبکه» هنگام نقاشی بسیار مفید است.

Peers			History		
Clear Log			Copy Selected		
[07:26:04]	RX	Up	shape	op=0902	
[07:26:13]	RX	Down-1	stroke	op=e3	
[07:26:16]	RX	Down-1	stroke	op=f7	
[07:26:27]	TX	Down-2	shape	op=d	
[07:26:29]	TX	Down-2	shape	op=1e	
[07:26:31]	TX	Down-2	shape	op=f7	
[07:26:33]	TX	Down-2	shape	op=1k	
[07:26:35]	TX	Down-2	shape	op=63	
[07:26:37]	TX	Down-2	shape	op=93	
[07:26:44]	TX	Down-2	shape	op=09	
[07:27:05]	RX	Up	text	op=031e5	
[07:27:10]	RX	Down-1	stroke	op=df	
[07:27:11]	RX	Down-1	undo	op=—	
[07:27:31]	RX	Down-1	text	op=8e8	
[07:27:34]	TX	Down-2	shape	op=ae	
[07:27:37]	TX	Down-2	undo	op=—	
[07:27:48]	TX	Down-2	text	op=41e	
[07:28:01]	TX	Down-2	shape	op=4f	
[07:28:13]	TX	Down-2	stroke	op=45	
[07:28:17]	TX	Down-2	stroke	op=a0	
[07:28:18]	TX	Down-2	undo	op=—	

Peers

History

Clear Log

Copy Selected

Up	shape op=09023d5b rect from
Down-1	stroke op=e3611dc0 pts=4
Down-1	stroke op=f796a8d3 pts=3
Down-2	shape op=d1ca2d93 line
Down-2	shape op=1e4f872a line
Down-2	shape op=f70ee657 line
Down-2	shape op=1be76678 line
Down-2	shape op=63258f82 line
Down-2	shape op=93ec0ea7 line
Down-2	shape op=09b65a8a oval
Up	text op=031e5b2a "from pc
Down-1	stroke op=df1eaab3 pts=2
Down-1	undo op=— target=df1e
Down-1	text op=8e8f4b08 "from
Down-2	shape op=aea10ba5 oval
Down-2	undo op=— target=aea1
Down-2	text op=41e580c5 "from
Down-2	shape op=4fcabd39f rect
Down-2	stroke op=4552d751 pts=3
Down-2	stroke op=a039e472 pts=4
Down-2	undo op=— target=a039

کد اصلی محاسبه نرخ‌ها:

```
sent_pkts, sent_bytes, recv_pkts, recv_bytes = self.transport.get_counters()
dt = max(0.001, t - self._last_stats_t)

sp = (sent_pkts - self._last_sent_pkts) / dt
sb = ((sent_bytes - self._last_sent_bytes) / 1024.0) / dt
rp = (recv_pkts - self._last_recv_pkts) / dt
rb = ((recv_bytes - self._last_recv_bytes) / 1024.0) / dt

self._send_rate = f"{sp:.1f} pkt/s, {sb:.1f} KB/s"
self._recv_rate = f"{rp:.1f} pkt/s, {rb:.1f} KB/s"
```

۱۲-۲ پشتیبانی اختیاری از IGMP با Scapy برای مشاهده‌پذیری بهتر در Wireshark

صورت مسئله فاز دوم الزام نکرده که IGMP حتماً دستی ساخته شود؛ کافی است برنامه به گروه ملحق شود. اما برای شفافیت در Wireshark و در برخی سناریوهای ویندوز، یک قابلیت اختیاری اضافه شده است:

- اگر برنامه با `igmp` اجرا شود، با Scapy پیام IGMPv2 Membership Report به صورت دوره‌ای ارسال می‌شود.

- هنگام خروج نیز پیام IGMPv2 Leave ارسال می‌گردد.

این کار دو فایده دارد:

- در Wireshark خیلی واضح می‌بینیم چه زمانی عضو شدیم/خارج شدیم.

- در برخی محیط‌ها عضویت IGMP پایدارتر می‌شود.

ساخت JOIN با Router Alert و TTL=1:

```
def _build_join(self):
    dst_mac = multicast_mac(self.group)
    return (
```

```
scapy_Ether(src=self.src_mac, dst=dst_mac) /
self._ip_with_router_alert(self.group) /
IGMP(type=0x16, gaddr=self.group)
)
```

و LEAVE:

```
def _build_leave(self):
leave_dst = "224.0.0.2"
dst_mac = multicast_mac(leave_dst)
return (
scapy_Ether(src=self.src_mac, dst=dst_mac) /
self._ip_with_router_alert(leave_dst) /
IGMP(type=0x17, gaddr=self.group)
)
```

تأکید: این بخش فقط **Plane Control** است؛ **Plane Data** برنامه (رویدادهای رسم) همچنان با socket انجام می‌شود تا دقیقاً مطابق نیازمندی فاز دوم باشد.

۱۳-۲ راهنمای اجرا و سناریوهای تست نهایی PIM و IGMP Proxy

۱-۱۳-۲ فرمان‌های نمونه برای اجرای برنامه روی هر میزبان

در هر میزبان، همان یک فایل اجرا می‌شود. تنها تفاوت، مقدار `--ifaddr` و `--user` است. نمونه:

```
# Upstream (PC1)
```

```
py mcast_canvas_paint.py --ifaddr 10.0.1.10 --user Up --group 239.1.1.1 --port 500
```

```
# Downstream (PC2)
```

```
py mcast_canvas_paint.py --ifaddr 10.0.2.10 --user Down-1 --group 239.1.1.1 --port
```

```
# Downstream (PC3)
```

```
py mcast_canvas_paint.py --ifaddr 10.0.2.11 --user Down-2 --group 239.1.1.1 --port
```

۲-۱۳-۲ فرمان‌های اجرای برنامه در دو حالت PIM-SM و IGMP Proxy

در این پروژه، کد برنامه می‌تواند در دو حالت عملیاتی اجرا شود. تفاوت اصلی این دو سناریو در نیاز (یا عدم نیاز) به گزینه‌ی peer-unicast است.

حالت PIM-SM مسیریابی واقعی Multicast؛ بدون نیاز به Unicast Assist:

PC1 - Upstream

```
py mcast_canvas.py --group 239.1.1.1 --port 5000 --ttl 16 --ifaddr 10.0.1.10 --use
```

PC2 - Downstream

```
py mcast_canvas.py --group 239.1.1.1 --port 5000 --ttl 16 --ifaddr 10.0.2.10 --use
```

PC3 - Downstream

```
py mcast_canvas.py --group 239.1.1.1 --port 5000 --ttl 16 --ifaddr 10.0.2.11 --use
```

حالت IGMP Proxy (مطابق صورت مسئله؛ با Unicast Assist برای مسیر برگشت):

PC1 - Upstream

```
python mcast_canvas.py --group 239.1.1.1 --port 5000 --ttl 16 --ifaddr 10.0.1.10
```

PC2 - Downstream

```
python mcast_canvas.py --group 239.1.1.1 --port 5000 --ttl 16 --ifaddr 10.0.2.10 -
```

PC3 - Downstream

```
python mcast_canvas.py --group 239.1.1.1 --port 5000 --ttl 16 --ifaddr 10.0.2.11 -
```

توضیح پارامترها:

- group 239.1.1.1: آدرس گروه چندپخش‌ی که همه‌ی میزبان‌ها عضو آن می‌شوند.
- port 5000: پورت UDP برای ارسال/دریافت رویدادها (باید روی همه یکی باشد).
- ttl 16: مقدار TTL برای بسته‌های Multicast؛ برای عبور از روتر باید ≥ 2 باشد.

- `<IP> ifaddr`:- آدرس IP اینترفیس محلی که Join و ارسال چندپخشی روی آن انجام می‌شود (برای جلوگیری از انتخاب کارت شبکه اشتباه).

- `"...-user`:- نام نمایشی کاربر در رابط گرافیکی و پیام‌های hello (اختیاری ولی مفید برای تست/تحلیل).

- `-igmp`:- فعال‌سازی ارسال IGMP JOIN/LEAVE با Scapy (اختیاری؛ فقط برای مشاهده‌پذیری بهتر در Wireshark).

- `-peer-unicast`:- مخصوص حالت IGMP Proxy؛ علاوه بر Multicast، پیام‌ها را به همتهای کشف‌شده به صورت Unicast هم می‌فرستد تا محدودیت مسیر $Downstream \rightarrow Upstream$.

نکته: در حالت PIM-SM معمولاً نیازی به `-peer-unicast` نیست، چون Multicast به صورت واقعی بین دو سگمنت route می‌شود.

نکته‌های عملی مهم:

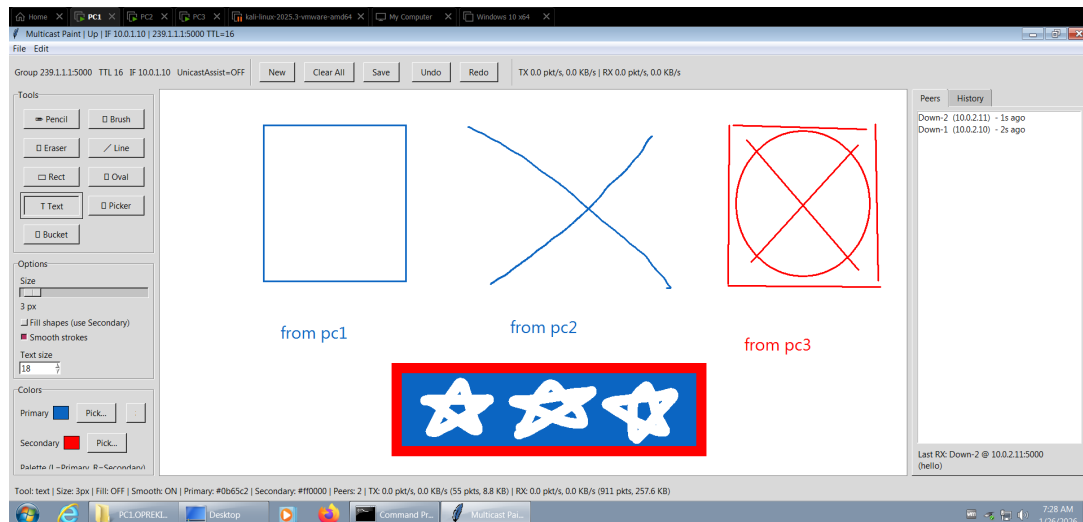
- روی ویندوز باید اجازه‌ی Inbound/Outbound UDP روی پورت (اینجا ۵۰۰۰) در Firewall داده شود.

- TTL باید حداقل ۲ باشد؛ ما از ۱۶ استفاده کردیم تا مطمئن باشد.

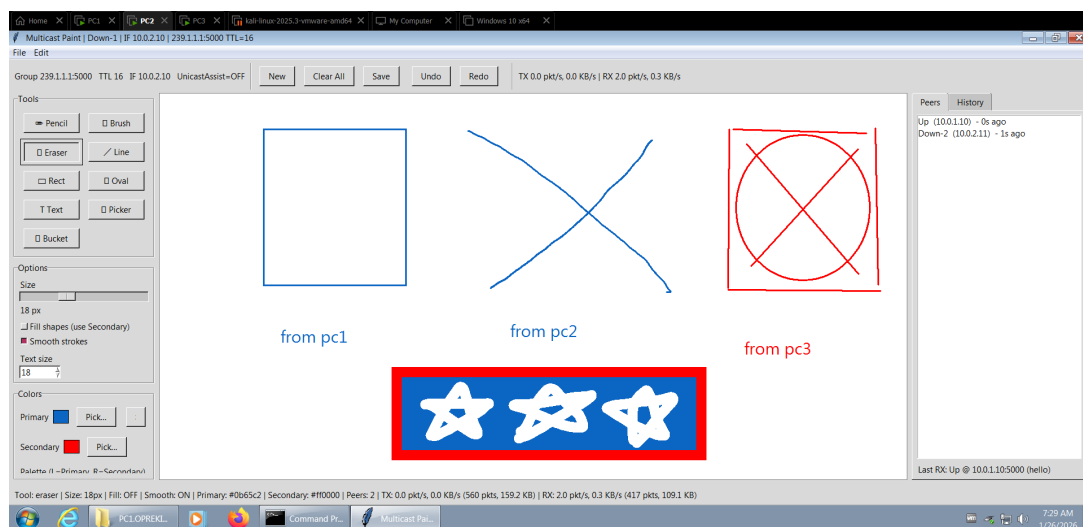
- در محیط‌هایی که Multicast دوطرفه نیست (مانند IGMP Proxy استاندارد)، بهتر است روی میزبان‌های Downstream گزینه‌ی `-peer-unicast` فعال باشد تا ارسال‌های آن‌ها در Upstream نیز دیده شود.

- در حالت PIM-SM معمولاً نیازی به `-peer-unicast` نیست؛ چون multicast بین سگمنت‌ها route می‌شود.

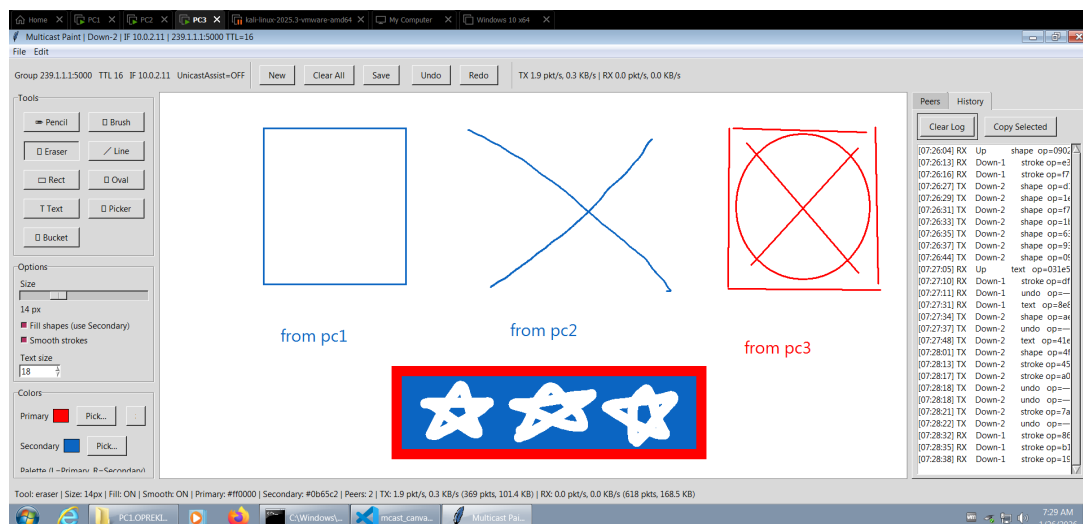
۲-۱۳-۳ خروجی برنامه در حالت PIM (Full Multicast Routing)



PC1 - Upstream



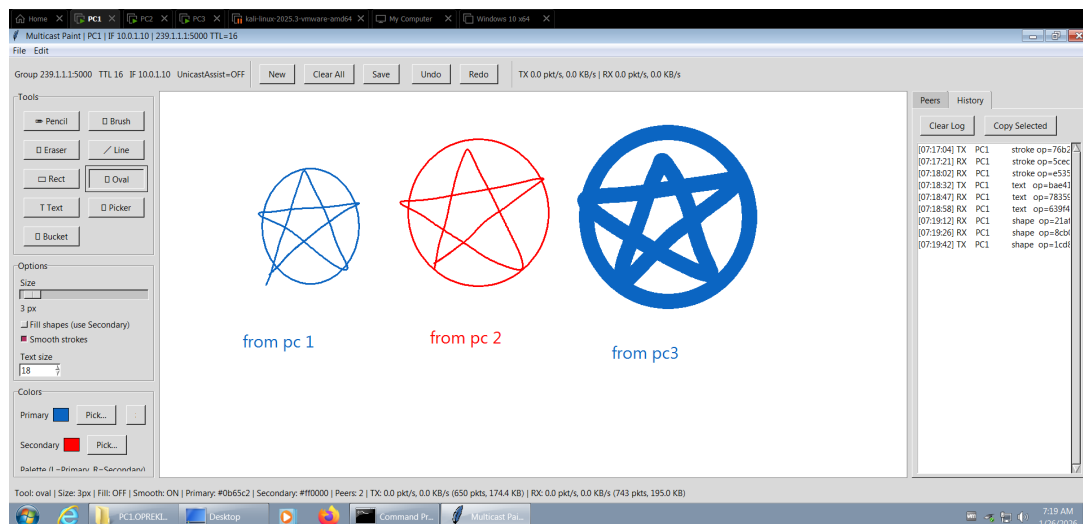
PC2 - Down-1



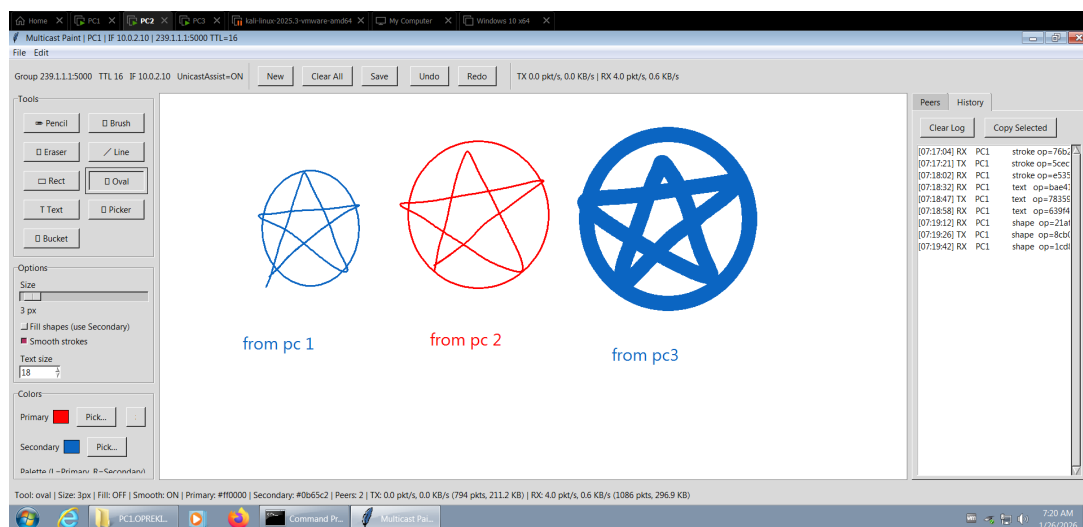
PC3 - Down-2

شکل ۹: اجرای برنامه در حالت PIM: همه کاربران خطوط یکدیگر را صرفاً با Multicast و در زمان واقعی مشاهده می‌کنند.

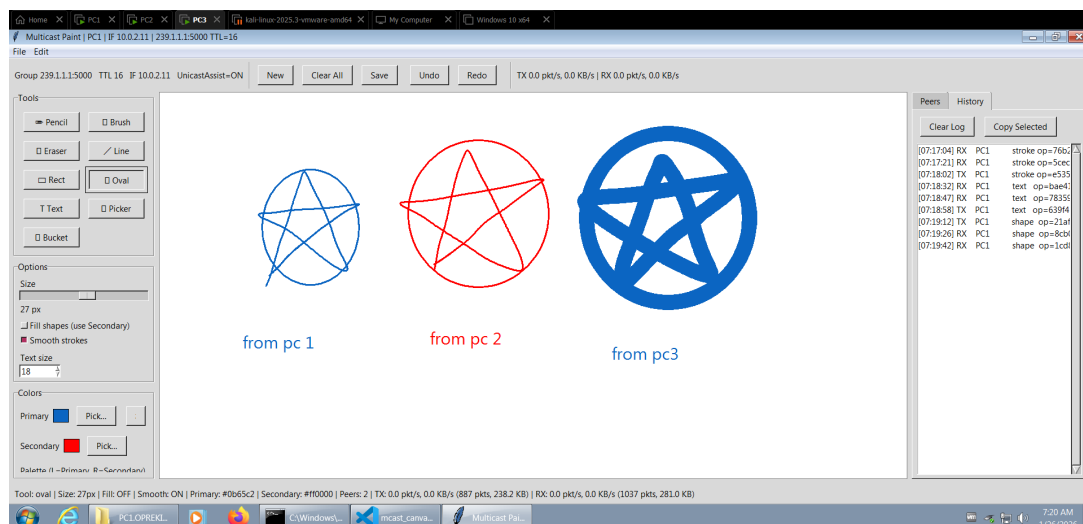
۲-۱۳-۴ خروجی برنامه در حالت Proxy IGMP (مطابق صورت مسئله)



PC1 - Upstream



PC2 - Down-1



PC3 - Down-2

شکل ۱۰: اجرای برنامه در حالت IGMP Proxy: مسیر Upstream->Downstream به صورت طبیعی Multicast است و برای مسیر برگشت از peer-unicast استفاده شده تا همگام‌سازی کامل شود.

۲-۱۳-۵ ابزار Torch روی MikroTik برای مشاهده زنده حرکت بسته‌ها

برای مشاهده‌ی زنده‌ی ترافیک UDP روی روتر، از دستور `/tool torch` استفاده شد.

```
[admin@MikroTik] > /tool torch interface=ether1 port=5000
```

```
[admin@MikroTik] > /tool torch interface=ether2 port=5000
```

تفسیر:

- ether1 سمت Downstream است؛ بنابراین در حالت IGMP Proxy بسته‌های Multicast خروجی به گیرنده‌ها روی اینترفیس واضح دیده می‌شود.
- ether2 سمت Upstream است؛ ترافیک بین PC1 و روتر و همچنین در حالت PIM یا در صورت عبور پیام‌ها به سمت Upstream در اینترفیس دیده می‌شود.

```

MikroTik RouterOS 7.19.4 (c) 1999-2025 https://www.mikrotik.com/

Press F1 for help

(20 messages not shown)
2026-01-23 09:43:08 system,error,critical router was rebooted without proper shutdown
2026-01-23 10:37:13 system,error,critical router was rebooted without proper shutdown
2026-01-23 11:20:17 system,error,critical router was rebooted without proper shutdown
2026-01-23 11:29:39 system,error,critical router was rebooted without proper shutdown
2026-01-23 13:00:29 system,error,critical router was rebooted without proper shutdown
2026-01-23 13:26:10 system,error,critical router was rebooted without proper shutdown
2026-01-24 17:26:08 system,error,critical router was rebooted without proper shutdown
2026-01-26 15:22:57 system,error,critical router was rebooted without proper shutdown

[admin@MikroTik] > /tool torch interface=ether1 port=5000

Columns: MAC-PROTOCOL, IP-PROTOCOL, DSCP, SRC-ADDRESS, SRC-PORT, DST-ADDRESS, DST-PORT, TX, RX, TX-PACKETS, RX-PACKETS
MAC-PROTOCOL IP-PROTOCOL DSCP SRC-ADDRESS SRC-PORT DST-ADDRESS DST-PORT TX RX TX-PACKETS RX-PACKETS
ip igmp 0 10.0.2.10 224.0.0.22 0bps 0bps 0 0
ip udp 0 10.0.2.11 5000 239.1.1.1 5000 0bps 0bps 0 0
ip udp 0 239.1.1.1 5000 10.0.1.10 5000 100.3kbps 0bps 35 0

[Q quit][D dump][C-z pause]

```

شکل ۱۱: خروجی Torch روی ether1 با فیلتر `port=5000`: مشاهده‌ی زنده‌ی ترافیک برنامه روی سمت Downstream.

```

MicroTic R1
MikroTik RouterOS 7.19.4 (c) 1999-2025 https://www.mikrotik.com/
Press F1 for help

(20 messages not shown)
2026-01-23 09:43:08 system,error,critical router was rebooted without proper shutdown
2026-01-23 10:37:13 system,error,critical router was rebooted without proper shutdown
2026-01-23 11:20:17 system,error,critical router was rebooted without proper shutdown
2026-01-23 11:29:33 system,error,critical router was rebooted without proper shutdown
2026-01-23 13:00:29 system,error,critical router was rebooted without proper shutdown
2026-01-23 13:26:10 system,error,critical router was rebooted without proper shutdown
2026-01-24 17:26:08 system,error,critical router was rebooted without proper shutdown
2026-01-26 15:22:57 system,error,critical router was rebooted without proper shutdown

[admin@Mikrotik] > /tool torch interface=ether1 port=5000
Columns: MAC-PROTOCOL, IP-PROTOCOL, DSCP, SRC-ADDRESS, SRC-PORT, DST-ADDRESS, DST-PORT, TX, RX, TX-PACKETS, RX-PACKETS
MAC-PROTOCOL IP-PROTOCOL DSCP SRC-ADDRESS SRC-PORT DST-ADDRESS DST-PORT TX RX TX-PACKETS RX-PACKETS
ip igmp 0 10.0.2.10 5000 239.1.1.1 5000 0bps 0bps 0 0
ip igmp 0 10.0.2.11 5000 239.1.1.1 5000 0bps 0bps 0 0
ip udp 0 10.0.2.10 5000 239.1.1.1 5000 0bps 0bps 0 0
ip udp 0 10.0.2.11 5000 239.1.1.1 5000 0bps 0bps 0 0

[admin@Mikrotik] > /tool torch interface=ether2 port=5000
Columns: MAC-PROTOCOL, IP-PROTOCOL, DSCP, SRC-ADDRESS, SRC-PORT, DST-ADDRESS, DST-PORT, TX, RX, TX-PACKETS, RX-PACKETS
MAC-PROTOCOL IP-PROTOCOL DSCP SRC-ADDRESS SRC-PORT DST-ADDRESS DST-PORT TX RX TX-PACKETS RX-PACKETS
ip udp 0 10.0.1.10 5000 239.1.1.1 5000 0bps 0bps 0 0
ip udp 0 239.1.1.1 5000 10.0.2.10 5000 329.9kpps 0bps 118 0
ip udp 0 239.1.1.1 5000 10.0.2.11 5000 0bps 0bps 0 0

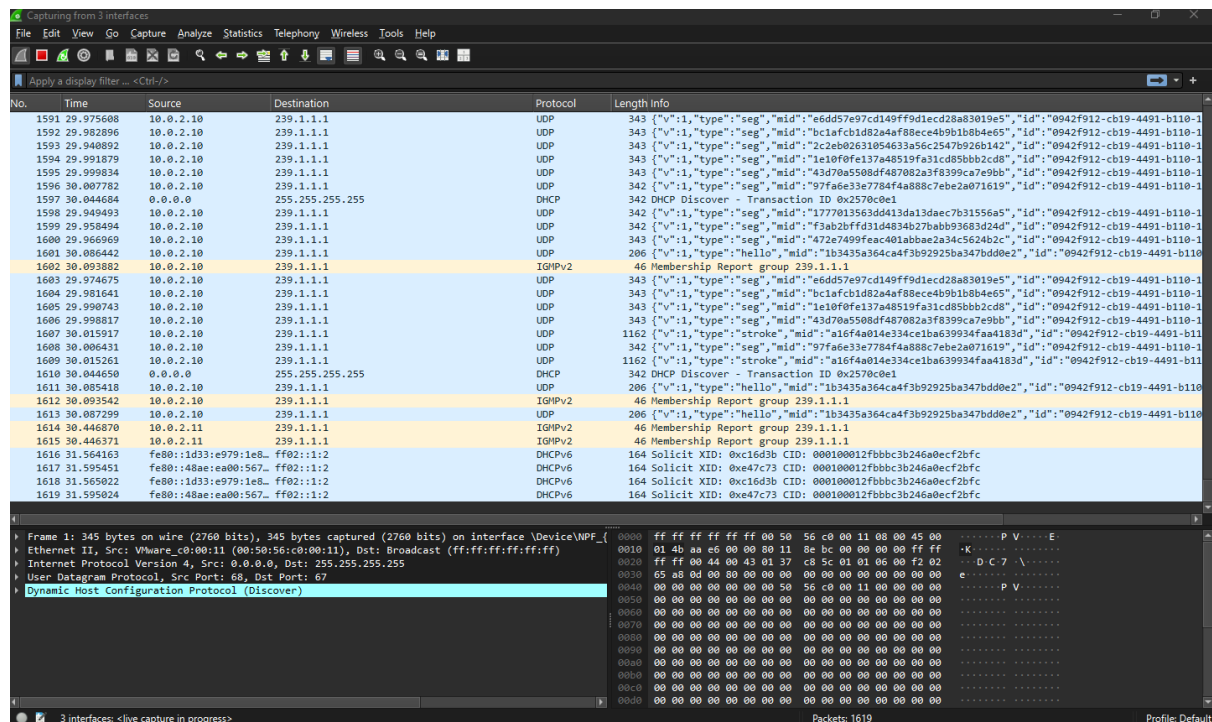
[0 quit][D dump][C-z pause]
solarwinds Solar-PuTTY free tool © 2019-2024 SolarWinds Worldwide, LLC. All rights reserved.

```

شکل ۱۲: خروجی Torch روی ether2 با فیلتر port=5000: مشاهده‌ی زنده‌ی ترافیک برنامه روی سمت Upstream.

۲-۱۴ تحلیل Wireshark: مشاهده‌ی واقعی پیام‌ها و ردیابی یک سناریو (بیضی پرشده با دورخط ثانویه)

برای اثبات عملی ارسال و دریافت رویدادهای برنامه روی شبکه، از Wireshark استفاده شد. در پیوست، فایل pcapng مربوط به ویدئوی نمونه نیز قرار داده شده است.



شکل ۱۳: نمونه‌ی Capture در Wireshark: بسته‌های UDP روی پورت ۵۰۰۰ به مقصد گروه 239.1.1.1 و پیام‌های کنترلی.

۲-۱۴-۱ فیلتر پیشنهادی و تفکیک ترافیک‌های نامرتب

برای اینکه ترافیک‌های نامرتب (مثل SSDP/UPnP) مزاحم تحلیل نشوند، این فیلتر مفید است:

```
udp.port == 5000 or igmp
```

نکته: در نمونه‌ی ارائه‌شده، فریم‌های ۵ و ۱۰ مربوط به SSDP هستند و به پروژه ارتباطی ندارند؛ بنابراین در تحلیل اصلی نادیده گرفته می‌شوند.

۲-۱۴-۲ تحلیل نمونه‌ی پیام hello (Presence) از دید بسته

در فریم‌های ۱ تا ۳ اطلاعات زیر دیده می‌شود:

- Dst IP برابر 239.1.1.1 و Dst Port برابر 5000 است (پس پیام Presence نیز روی همان کانال چندپخش‌ی است).

- Dst MAC برابر 01:00:5e:01:01:01 است که نگاشت استاندارد 239.1.1.1 روی Ethernet است.

• داده‌ی UDP payload یک رشته‌ی JSON با type="hello" است.

نمونه‌ی متن Payload (فریم ۱):

```
{"v":1,"type":"hello","mid":"...","id":"...","user":"Down-1","ts":...,"port":5000,
```

نکته: وجود پیام‌های hello باعث می‌شود حتی اگر کاربری در حال رسم نباشد، همتاها از حضور او مطلع بمانند و برنامه بتواند peer list را live به‌روزرسانی کند. همچنین در حالت peer-unicast همین مکانیسم باعث یادگیری IP همتاها می‌شود.

۲-۱۴-۳ تحلیل Join IGMP ساخته‌شده توسط Scapy

در فریم ۷ یک بسته‌ی IGMP دیده می‌شود:

• Protocol=IGMP و TTL=1

• وجود IP Option: Router Alert (به همین دلیل Header Length=24 شده است)

• Src IP=10.0.1.10 و Dst IP=239.1.1.1 که با JOIN فاز دوم (در حالت -igmp) مطابق است.

نکته اندازه‌ای:

فریم ۷ در Wireshark برابر 46 bytes گزارش شده است. این مقدار دقیقاً از جمع زیر به دست می‌آید:

$$\text{Ethernet}(14) + \text{IP}(24 \text{ Alert Router}) + \text{IGMP}(8) = 46$$

۲-۱۴-۴ ردیابی سناریوی خواسته‌شده: رسم یک بیضی پرشده با دورخط ثانویه و تاخیر Release

طبق درخواست صورت‌مسئله و سناریوی تست، هدف این است که نشان دهیم:

• هنگام «کشیدن» Shape هیچ پیام شبکه‌ای ارسال نمی‌شود و فقط Preview محلی داریم؛

• در لحظه‌ی رها کردن ماوس (ButtonRelease)، یک پیام shape ارسال می‌شود؛

• این پیام شامل fill=true و outline (برای دورخط ثانویه) و fill_color (برای رنگ پرکردن) است.

خلاصه‌ی فریم‌های کلیدی از خروجی ارائه‌شده:

نقش در سناریو	Payload UDP	TTL	Type	IP Src	Frame
حضور Down-1 (رسیده به سمت دیگر)	۱۶۴B	۱۵	hello	۱۰.۲.۰.۱۰	۱
حضور Down-2	۱۶۴B	۱۵	hello	۱۱.۲.۰.۱۰	۲
حضور Up	۱۶۰B	۱۶	hello	۱۰.۱.۰.۱۰	۳
Join ساخته شده توسط Scapy	—	۱	IGMP	۱۰.۱.۰.۱۰	۷
ارسال Shape در لحظه Release	۳۲۶B	۱۵	shape	۱۱.۲.۰.۱۰	۹

جدول ۲: خلاصه فریم‌های کلیدی سناریوی Wireshark ارائه شده توسط پروژه.

تفسیر تاخیر (Delay) در ایجاد Shape:

در خروجی Wireshark، فاصله‌ی زمانی بین فریم ۸ و فریم ۹ حدود 1.8s گزارش شده است. این بازه دقیقاً همان زمانی است که کاربر در حال «کلیک و کشیدن» Shape بوده است. چون Preview فقط محلی است، در شبکه چیزی دیده نمی‌شود تا زمانی که کاربر ماوس را رها کند؛ در همان لحظه پیام shape ساخته و ارسال می‌شود.

تحلیل دقیق اندازه‌ی فریم ۹ (پیام shape):

Wireshark برای فریم ۹ مقادیر زیر را نشان می‌دهد:

• 368 bytes = Frame length

• 354 bytes = IP total length

• 334 bytes = UDP length

• 326 bytes = UDP payload

این اعداد دقیقاً با ساختار استاندارد Ethernet + IPv4 + UDP سازگار است:

$$\text{Ethernet}(14) + \text{IP}(20) + \text{UDP}(8) + \text{Payload}(326) = 368$$

تحلیل TTL و اثبات عبور از روتر:

در فریم ۹ مقدار TTL=15 دیده می‌شود، در حالی که برنامه TTL=16 تنظیم کرده است. بنابراین TTL=15 نشان می‌دهد بسته پس از عبور از یک روتر یک واحد کم شده و این همان چیزی است که در توپولوژی دو-سگمندی پروژه انتظار داریم.

تحلیل فیلدهای کاربردی پیام shape در فریم ۹:

طبق متن JSON دیده شده در فریم ۹:

• type="shape" و shape="oval": یعنی ابزار بیضی

• p0 و p1: دو گوشه‌ی نرمال شده بیضی (مبدأ و مقصد Drag)

• op: شناسه عملیات برای Undo/Redo

• mid: شناسه یکتا برای Dedup

برای حالت Fill و دورخط ثانویه:

در اجرای واقعی پروژه، چون گزینه Fill shapes فعال بوده است، در JSON فریم ۹ باید این فیلدها نیز وجود داشته باشند:

• fill:true

• outline:"..." (رنگ Secondary برای دورخط)

• fill_color:"..." (رنگ Primary برای پرکردن)

اگر Wireshark متن JSON را کوتاه نمایش می‌دهد، کافی است روی Data راست کلیک کنید و Copy -> ... as printable text بزنید یا بخش Packet Bytes را باز کنید تا رشته کامل JSON دیده شود.

۲-۱۵ بررسی تحقق نیازمندی‌های فاز دوم (چک لیست نهایی)

در این بخش، دقیقاً بندهای صورت مسئله‌ی فاز دوم با پیاده‌سازی انجام شده تطبیق داده می‌شود.

• برنامه‌ی واحد ارسال/دریافت (Single Send/Recv): Both Program

فایل mcast_canvas_paint.py روی همه‌ی میزبان‌ها اجرا می‌شود و همزمان هم ارسال و هم دریافت را انجام می‌دهد (بدون سرور).

• استفاده از UDP + Multicast با Group/Port/TTL قابل تنظیم:

پارامترهای -group -port -ttl در CLI وجود دارند و در کلاس MulticastTransport اعمال شده‌اند.

• ارسال رویدادهای Canvas به صورت UDP: Multicast

برای ترسیم‌ها پیام‌های seg/stroke/shape/text/bg/clear/undo به صورت JSON ساخته و با sendto ارسال می‌شوند.

• دریافت و Parse و رسم در Canvas

در حلقه‌ی دریافت شبکه، هر پیام JSON دریافت و بر اساس type هندل می‌شود:

- seg برای نمایش زنده
- stroke/shape/text برای رسم نهایی
- bg/clear/undo برای عملیات مدیریتی

• نمایش Canvas مشترک و مشاهده رویدادها در زمان واقعی:

در تست‌های ارائه‌شده (تصاویر حالت PIM و IGMP Proxy) هر سه میزبان تغییرات را همزمان می‌بینند.

• اجرای نهایی روی Upstream و Downstream و عبور از روتر:

با توجه به توپولوژی پروژه و استفاده از TTL=16، عبور از روتر اتفاق می‌افتد و هم با Torch و هم با Wireshark قابل مشاهده است.

• پشتیبانی از Proxy IGMP و حل سناریوی دوطرفه:

برای اینکه در حالت IGMP Proxy نیز مسیر برگشت برقرار باشد، گزینه‌ی peer-unicast- اضافه شده که بدون حذف، Multicast مسیر برگشت را با Unicast تکمیل می‌کند.

• اجرای پیشرفته با PIM-SM روی RouterOS v7:

IGMP Proxy حذف و PIM-SM مطابق داکيومنت Mikrotik فعال شده است^۶ تا multicast بین دو سگمنت به صورت واقعی route شود.

• افزونه‌ی اختیاری IGMP با Scapy برای رویت‌پذیری بیشتر:

گزینه‌ی igmp- امکان ارسال دوره‌ای JOIN و ارسال LEAVE هنگام خروج را فراهم کرده است (صرفاً کمکی و آموزشی).

۱۶-۲ جمع‌بندی نهایی

در فاز دوم، یک برنامه‌ی کامل و تعاملی Canvas اشتراکی پیاده‌سازی شد که:

- روی هر میزبان تنها با اجرای یک فایل کار می‌کند؛
- بدون سرور و با استفاده از UDP Multicast همگام‌سازی می‌شود؛

^۶[help.mikrotik.com – PIM-SM \(RouterOS\)](https://help.mikrotik.com – PIM-SM (RouterOS))

• قابلیت‌های کاربردی مشابه Windows Paint (ابزارهای متعدد، رنگ‌های Pri- mary/Secondary، پرکردن شکل، متن، پاک‌کن، Undo/Redo، ذخیره خروجی و ...) دارد؛

• از نظر شبکه‌ای قابل تحلیل است (هم در سطح Socket و هم در سطح IGMP با Scapy)؛
• در دو حالت عملیاتی IGMP Proxy و PIM-SM تست شده و خروجی‌های واقعی (تصویر، Torch، Wireshark) به عنوان شاهد آورده شده‌اند؛

• و از نظر مهندسی شبکه، مسئله‌ی «many-to-many» «multicast» را در IGMP Proxy با peer-unicast و در حالت درست‌تر با PIM-SM حل کرده است.

۱۷-۲ پیوست‌ها (Appendix)

۱-۱۷-۲ پیوست A: کد فاز اول نسخه Scapy

کد کامل فاز اول در فایل‌های زیر در کنار این گزارش (پوشه پروژه) موجود است:

• Phase1_Scapy_Sender.py

• Phase1_Scapy_Receiver.py

۲-۱۷-۲ پیوست B: کد کامل فاز دوم

کد کامل برنامه‌ی اصلی فاز دوم در فایل زیر موجود است:

• mcast_canvas_paint.py

این فایل شامل:

• کلاس MulticastTransport برای UDP Multicast

• کلاس اختیاری ScapyIGMPKeepalive برای IGMP

• کلاس MulticastPaintApp برای UI، همگام‌سازی، Undo/Redo و رندر

۳-۱۷-۲ پیوست C: فایل ویدیوی کوتاه و فایل Wireshark

برای نمایش بهتر عملکردهای Paint-like (ابزارها، Undo/Redo، متن، Fill، تغییر پس‌زمینه و...) یک ویدیوی کوتاه از اجرای همزمان روی سه میزبان تهیه شده و در کنار این گزارش قرار داده شده است. همچنین فایل Wireshark capture (فرمت pcapng) مربوط به همان ویدئو نیز ضمیمه شده است تا داور بتواند پیام‌ها را دقیقاً بررسی کند.